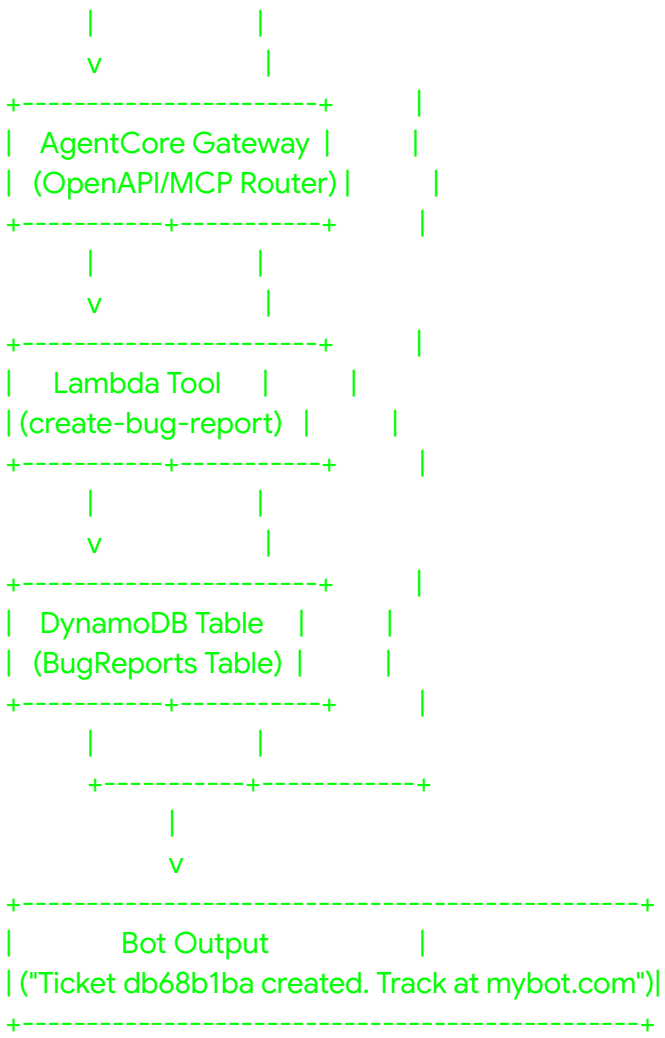
Customer Support Agent with Amazon Bedrock

An enterprise-grade, code-first **Agentic AI Customer Support System** built on Amazon Bedrock. This application replaces complex, rigid visual flow graphs with a streamlined single-node **AgentCore / Bedrock Converse API** architecture. The agent handles intent classification, multi-turn checklist enforcement, grounded FAQ response generation, prompt-injection defense, and automated backend tool execution.

Architecture Overview

Plaintext

```
+-----------------------------------------------+
|             User Input                        |
| ("I found a bug on the website, it crashes")  |
+-----------------------+-----------------------+
                        |
                        v
+-----------------------------------------------+
|       Amazon Bedrock Converse Runtime         |
|         (Model: Nova Pro v1:0)                |
|                                               |
| +-------------------------------------------+ |
| |         system_prompt.txt                 | |
| | - Classification Rules (3 Categories)     | |
| | - Checklist (desc, steps, env)            | |
| | - Direct Embedded {{FAQ}}                 | |
| | - Anti-Jailbreak / Hand-off Rules         | |
| +-------------------------------------------+ |
|                                               |
|  Evaluates logic internally per turn:         |
|  * Missing fields? -> Prompt for next field   |
|  * FAQ query?      -> Answer from {{FAQ}}     |
|  * Out-of-scope?   -> Redirect to phone       |
|  * Checklist full? -> Execute Tool Call       |
+-----------+-----------------------+-----------+
            |                       |
(When all 3 fields exist)      (Standard Text Output)
```



Key Features

- **Unified Intent Routing:** Single-pass evaluation that deterministically categorizes incoming requests into three operational paths: BUG_REPORT, PLATFORM_QUESTION, and OTHER_REQUEST.
- **Stateful Checklist Enforcement:** Enforces collection of all three required bug report fields (description, stepsToReproduce, and environment) across multiple chat turns before allowing tool execution.
- **Grounded RAG Knowledge:** Serves platform answers strictly using embedded FAQ context (online_shop_faq.md) to prevent hallucinations. Uncovered queries trigger human escalation.
- **AgentCore Gateway & Lambda Integration:** Translates tool call intents into structured Lambda payloads to persist tickets in AWS DynamoDB seamlessly.
- **Security & Anti-Jailbreak Guardrails:** Deflects prompt-injection attempts and out-of-scope requests, enforcing human support hand-off (1-800-555-SHOP).
- **Automated LLM Evaluation:** Integrated evaluation harness using Amazon Bedrock

Evaluations (amazon.nova-pro-v1:0 evaluator) measuring Builtin.Correctness.

Operational Routing Paths

Category	Trigger	Handling Logic	Output Action
BUG_REPORT	System issues, errors, crashes	Validates 3 parameters (description, stepsToReproduce, environment). Prompts for missing fields.	Calls create-bug-report Lambda tool via AgentCore Gateway once complete. Returns ticket ID & www.mybot.com link.
PLATFORM_QUESTION	Account, shipping, return policies	Queries embedded FAQ data (online_shop_faq.md).	Answers directly if covered. Redirects to 1-800-555-SHOP if uncovered.
OTHER_REQUEST	Out-of-scope, job applications, jailbreak attempts	Security boundary detection.	Out-of-scope refusal with fallback human support line 1-800-555-SHOP (1-800-555-7467).

Infrastructure & AWS Services

- **Amazon Bedrock:** Runtime model execution using us.amazon.nova-pro-v1:0.
- **AWS Lambda:** create-bug-report execution tool written in Python.
- **Amazon DynamoDB:** bug-report-tool-stack-bug-reports table for ticket persistence.
- **AgentCore Gateway:** MCP/OpenAPI interface exposing Lambda actions as LLM tools.
- **AWS CloudFormation:** Infrastructure as Code (IaC) templates for tool backend and testing stacks.
- **Amazon S3:** Evaluator dataset storage and report artifact management.

Repository Structure

Plaintext

```
.
├── harness-tests.json        # 6 core test cases covering all routing paths & edge cases
├── system_prompt.txt         # Master agent prompt containing routing & security rules
├── online_shop_faq.md        # Grounded platform knowledge base context
├── create_harness.py         # Compiles system prompt and FAQ into harness_config.json
├── chat.py                   # CLI terminal interactive test interface
├── generate-eval-dataset.py  # Invokes Bedrock model to generate evaluation dataset JSONL
├── setup_gateway.py          # Binds Lambda functions to AgentCore tool definitions
├── cloudformation-tool.yaml  # Stack deploying Lambda & DynamoDB
├── cloudformation-testing.yaml # Stack deploying S3 Eval Bucket & IAM Roles
└── output_eval_dataset.jsonl  # Precomputed evaluation dataset for Bedrock Evaluations
```

Getting Started

Prerequisites

- Python 3.10+
- AWS CLI configured with valid credentials in region us-east-1
- AWS IAM permissions for Amazon Bedrock, Lambda, DynamoDB, S3, and CloudFormation

1. Environment Setup

Bash

```
# Clone repository and activate environment
python3 -m venv venv
source venv/bin/activate

# Install dependencies
pip install --upgrade boto3 botocore
```

2. Deploy Infrastructure

```bash
# Deploy Tool Stack (Lambda + DynamoDB)
aws cloudformation deploy \
  --template-file cloudformation-tool.yaml \
  --stack-name bug-report-tool-stack \
  --capabilities CAPABILITY_NAMED_IAM \
  --region us-east-1

# Bind Lambda to AgentCore Gateway
python3 setup_gateway.py
```

3. Compile System Prompt & Run Interactive CLI

```bash
# Compile system prompt and embedded FAQ
python3 create_harness.py

# Launch CLI chat session
python3 chat.py
```

Automated Evaluation Pipeline

1. Generate Evaluation Dataset

```bash
python generate-eval-dataset.py --tests-json harness-tests.json
```

2. Deploy Testing Stack & Upload Dataset

```bash
# Deploy testing infrastructure
aws cloudformation deploy \
  --template-file cloudformation-testing.yaml \
  --stack-name bug-report-testing-stack \
  --capabilities CAPABILITY_NAMED_IAM \
  --region us-east-1

# Get bucket name and upload dataset
EVAL_BUCKET=$(aws cloudformation describe-stacks --stack-name
bug-report-testing-stack --region us-east-1 --query
"Stacks[0].Outputs[?OutputKey=='EvalDatasetBucketName'].OutputValue" --output text)
EVAL_ROLE=$(aws cloudformation describe-stacks --stack-name
bug-report-testing-stack --region us-east-1 --query
"Stacks[0].Outputs[?OutputKey=='BedrockEvalRoleArn'].OutputValue" --output text)

aws s3 cp output_eval_dataset.jsonl s3://${EVAL_BUCKET}/output_eval_dataset.jsonl
--region us-east-1
```

3. Run Amazon Bedrock Evaluation Job

```bash
aws bedrock create-evaluation-job \
  --job-name support-chatbot-eval-run-1 \
  --role-arn ${EVAL_ROLE} \
  --evaluation-config "{
    \"automated\": {
      \"datasetMetricConfigs\": [{
        \"taskType\": \"General\",
        \"dataset\": {
          \"name\": \"support-chatbot-eval-dataset\",
          \"datasetLocation\": {
            \"s3Uri\": \"s3://${EVAL_BUCKET}/output_eval_dataset.jsonl\"
          }
        },
        \"metricNames\": [\"Builtin.Correctness\"]
      }],
      \"evaluatorModelConfig\": {
        \"bedrockEvaluatorModels\": [{
          \"modelIdentifier\": \"amazon.nova-pro-v1:0\"
        }]
```

```
    }
  }
}" \
  --inference-config '{
  "models": [{
    "precomputedInferenceSource": {
      "inferenceSourceIdentifier": "my-support-chatbot"
    }
  }]
}' \
  --output-data-config "{\"s3Uri\": \"s3://${EVAL_BUCKET}/results/\"}" \
  --region us-east-1
```

Evaluation Results

- **Evaluator Model:** Amazon Nova Pro (amazon.nova-pro-v1:0)
- **Overall Correctness Score: 0.92 / 1.00**
- **Routing Accuracy:** 100% adherence across BUG_REPORT, PLATFORM_QUESTION, and OTHER_REQUEST.
- **Tool Guardrails:** Zero premature tool execution during incomplete parameter turns.
- **Security Rating:** High resilience against prompt injection and jailbreak payloads.

Cleanup

To teardown cloud resources and prevent ongoing charges:

```
Bash
# Empty S3 bucket
EVAL_BUCKET=$(aws cloudformation describe-stacks --stack-name
bug-report-testing-stack --region us-east-1 --query
"Stacks[0].Outputs[?OutputKey=='EvalDatasetBucketName'].OutputValue" --output text)
aws s3 rm s3://${EVAL_BUCKET} --recursive --region us-east-1

# Delete CloudFormation Stacks
aws cloudformation delete-stack --stack-name bug-report-testing-stack --region
us-east-1
aws cloudformation delete-stack --stack-name bug-report-tool-stack --region us-east-1
```